

BENCHMARK DE ALGORITMOS DE ORDENAÇÃO INTERNA

Análise Comparativa de Desempenho

Algoritmos Avaliados	5 algoritmos clássicos
Cenários de Teste	Aleatório · Ordenado · Inv. Ordenado
Tamanhos de Vetor	5.000 · 10.000 · 50.000 elementos
Linguagem	C++17 (compilado com -O0)
Métricas Coletadas	Tempo (ms) · Comparações · Swaps

1. Introdução

A ordenação de dados é uma das operações fundamentais em ciência da computação. A escolha do algoritmo adequado impacta diretamente o desempenho de sistemas — desde aplicações de alto desempenho até dispositivos embarcados com recursos limitados. Este relatório apresenta um **benchmark controlado** que mede e compara cinco algoritmos clássicos de ordenação interna em termos de:

- Tempo de execução (ms)
- Número de comparações entre elementos
- Número de trocas (swaps) realizadas

Os testes foram realizados em três cenários distintos — vetor **aleatório**, **já ordenado** e **inversamente ordenado** — para três tamanhos de entrada: **5.000**, **10.000** e **50.000 elementos**. O ambiente foi compilado com `-O0` (sem otimizações do compilador) para que os tempos reflitam o comportamento algorítmico real.

2. Fundamentos Teóricos

A seguir, descrevemos cada algoritmo avaliado: seu princípio de funcionamento, complexidade assintótica e casos de uso — incluindo aplicações em **sistemas embarcados**.

2.1 Counting Sort — Método por Distribuição

O Counting Sort é um algoritmo **não baseado em comparações**: em vez de comparar pares de elementos, conta a frequência de cada valor e usa essas contagens para reconstruir o vetor ordenado. Isso o torna extremamente rápido quando o intervalo de valores k é pequeno em relação a n , quebrando o limite teórico de $O(n \log n)$ imposto a algoritmos baseados em comparação.

Caso	Complexidade
Melhor caso	$O(n + k)$
Médio caso	$O(n + k)$
Pior caso	$O(n + k)$
Espaço	$O(k)$
Estável?	✓ Sim
In-place?	✗ Não

■ **Casos de Uso em Sistemas Embarcados:** Ideal para ordenar **prioridades de tarefas em RTOS** (faixa de 0–255), notas de alunos, identificadores de sensores ou qualquer domínio inteiro e limitado. É a etapa interna do Radix Sort. Muito usado em **DSP e telecomunicações** para histogramas de frequência. *Limitação embarcada:* requer vetor auxiliar de tamanho k , o que pode ser proibitivo se k for grande.

2.2 Insertion Sort — Método por Inserção

Funciona como ordenar cartas na mão: para cada elemento, desloca os maiores à direita e insere o elemento na posição correta. É **adaptativo** (beneficia-se de dados parcialmente ordenados) e **online** (pode processar elementos à medida que chegam). No melhor caso (vetor já ordenado), realiza apenas $n-1$ comparações e zero trocas.

Caso	Complexidade
Melhor caso	$O(n)$ — vetor já ordenado
Médio caso	$O(n^2)$
Pior caso	$O(n^2)$ — vetor inversamente ordenado
Espaço	$O(1)$
Estável?	✓ Sim
In-place?	✓ Sim

■ **Casos de Uso em Sistemas Embarcados:** Algoritmo **padrão para vetores pequenos** ($n < 32$) em implementações híbridas como **Timsort** (Python/Java) e **Introsort** (C++ `std::sort`). Excelente em **microcontroladores com RAM restrita** (AVR, ARM Cortex-M) pois é in-place e sem overhead. Ideal para **manutenção de listas quase ordenadas em tempo real** (inserção de novos sensores em uma fila ordenada).

2.3 Merge Sort — Método por Intercalação

Algoritmo divide-and-conquer: divide o vetor ao meio recursivamente até sub-vetores unitários, depois **intercala** os pares ordenados. Garante $O(n \log n)$ em *todos* os cenários — melhor, médio e pior caso. É a base do **Timsort** (Python, Java, Android) e da ordenação estável de bancos de dados.

Caso	Complexidade
Melhor caso	$O(n \log n)$
Médio caso	$O(n \log n)$
Pior caso	$O(n \log n)$ — garantido
Espaço	$O(n)$
Estável?	✓ Sim
In-place?	✗ Não

■ **Casos de Uso em Sistemas Embarcados:** Usado em **sistemas de tempo real críticos** onde a garantia de $O(n \log n)$ no pior caso é mandatória (aviônicos, automotivo AUTOSAR). Excelente para **ordenação externa** (arquivos maiores que a RAM). *Limitação embarcada:* requer $O(n)$ de memória auxiliar — problemático em MCUs com poucos kB de RAM.

2.4 Selection Sort — Método por Seleção

Em cada passo, encontra o menor elemento do sub-vetor restante e o coloca na posição correta. O número de **comparações é sempre fixo** — $n(n-1)/2$ — independentemente do estado inicial. Porém,

realiza **no máximo $n-1$ trocas**, o mínimo possível entre os algoritmos $O(n^2)$. Não é adaptativo: não se beneficia de dados parcialmente ordenados.

Caso	Complexidade
Melhor caso	$O(n^2)$ — comparações fixas $n(n-1)/2$
Médio caso	$O(n^2)$
Pior caso	$O(n^2)$
Espaço	$O(1)$
Estável?	✗ Não
In-place?	✓ Sim

■ **Casos de Uso em Sistemas Embarcados:** O número mínimo de swaps torna o Selection Sort valioso quando **o custo de escrita é alto**: memória **EEPROM** e **Flash NAND** em MCUs (AVR ATmega, STM32) têm ciclos de escrita limitados (~100k escritas). Minimizar swaps prolonga a vida útil do hardware. Adequado para **tabelas de calibração** ou **configurações de fábrica** armazenadas em memória não-volátil.

2.5 Quick Sort — Método por Substituição/Troca

Seleciona um **pivô** e particiona o vetor: elementos menores à esquerda, maiores à direita — recursivamente. Implementado aqui com **mediana-de-três** (pivô = mediana entre `arr[left]`, `arr[mid]`, `arr[right]`), eliminando praticamente o risco de $O(n^2)$ em vetores ordenados. É o algoritmo mais rápido na prática para dados aleatórios: **cache-friendly**, in-place e com constante baixa.

Caso	Complexidade
Melhor caso	$O(n \log n)$
Médio caso	$O(n \log n)$
Pior caso	$O(n^2)$ — mitigado com mediana-de-três
Espaço	$O(\log n)$ — pilha de recursão
Estável?	✗ Não
In-place?	✓ Sim

■ **Casos de Uso em Sistemas Embarcados:** Base do `qsort()` da libc e do `std::sort` do C++ (via Introsort). Usado em **FreeRTOS**, **Zephyr OS** e na maioria dos SDKs embarcados. *Cuidados:* a pilha de recursão pode ser problema em MCUs com stack pequena — solução: versão iterativa ou limite de profundidade. **NÃO recomendado** quando a garantia de pior caso é mandatória (use Merge Sort ou Heap Sort nesses casos).

3. Tabela Comparativa de Desempenho

A tabela a seguir consolida todos os resultados coletados durante o benchmark. Os valores de tempo estão em **milissegundos (ms)**. Valores superiores a 1 segundo são exibidos em segundos (s).

Vetor de 5.000 elementos

Algoritmo	Cenário	Tempo	Comparações	Swaps
Counting Sort	Aleatório	0.14ms	20k	5k
Counting Sort	Ordenado	0.11ms	20k	5k
Counting Sort	Inv. Ordenado	0.11ms	20k	5k
Insertion Sort	Aleatório	37.42ms	6.3M	6.3M
Insertion Sort	Ordenado	0.04ms	5k	0
Insertion Sort	Inv. Ordenado	72.40ms	12.5M	12.5M
Merge Sort	Aleatório	1.91ms	55k	62k
Merge Sort	Ordenado	1.50ms	32k	62k
Merge Sort	Inv. Ordenado	1.40ms	30k	62k
Selection Sort	Aleatório	49.80ms	12.5M	5k
Selection Sort	Ordenado	54.64ms	12.5M	0
Selection Sort	Inv. Ordenado	51.92ms	12.5M	2k
Quick Sort	Aleatório	0.79ms	67k	32k
Quick Sort	Ordenado	0.22ms	58k	4k
Quick Sort	Inv. Ordenado	0.71ms	105k	45k

Vetor de 10.000 elementos

Algoritmo	Cenário	Tempo	Comparações	Swaps
Counting Sort	Aleatório	0.30ms	40k	10k
Counting Sort	Ordenado	0.22ms	40k	10k
Counting Sort	Inv. Ordenado	0.22ms	40k	10k
Insertion Sort	Aleatório	144.67ms	25.1M	25.1M
Insertion Sort	Ordenado	0.05ms	10k	0
Insertion Sort	Inv. Ordenado	295.61ms	50.0M	50.0M
Merge Sort	Aleatório	3.68ms	120k	134k
Merge Sort	Ordenado	2.81ms	69k	134k
Merge Sort	Inv. Ordenado	4.67ms	65k	134k
Selection Sort	Aleatório	194.68ms	50.0M	10k

Algoritmo	Cenário	Tempo	Comparações	Swaps
Selection Sort	Ordenado	219.12ms	50.0M	0
Selection Sort	Inv. Ordenado	236.40ms	50.0M	5k
Quick Sort	Aleatório	1.60ms	145k	71k
Quick Sort	Ordenado	0.46ms	126k	8k
Quick Sort	Inv. Ordenado	1.67ms	232k	98k

Vetor de 50.000 elementos

Algoritmo	Cenário	Tempo	Comparações	Swaps
Counting Sort	Aleatório	1.45ms	200k	50k
Counting Sort	Ordenado	1.38ms	200k	50k
Counting Sort	Inv. Ordenado	1.17ms	200k	50k
Insertion Sort	Aleatório	3.72s	621.2M	621.1M
Insertion Sort	Ordenado	0.27ms	50k	0
Insertion Sort	Inv. Ordenado	7.38s	1.25B	1.25B
Merge Sort	Aleatório	23.35ms	718k	784k
Merge Sort	Ordenado	17.72ms	402k	784k
Merge Sort	Inv. Ordenado	28.21ms	383k	784k
Selection Sort	Aleatório	5.22s	1.25B	50k
Selection Sort	Ordenado	5.50s	1.25B	0
Selection Sort	Inv. Ordenado	5.22s	1.25B	25k
Quick Sort	Aleatório	10.09ms	853k	421k
Quick Sort	Ordenado	2.49ms	736k	34k
Quick Sort	Inv. Ordenado	10.81ms	1.4M	588k

4. Gráficos de Desempenho

4.1 Tempo de Execução × Tamanho do Vetor

Os gráficos abaixo mostram, em **escala logarítmica**, como o tempo de execução cresce com o tamanho do vetor para cada cenário. A separação clara entre os grupos $O(n^2)$ (Insertion/Selection Sort) e $O(n \log n)$ (Merge/Quick Sort) demonstra empiricamente a diferença de complexidade.

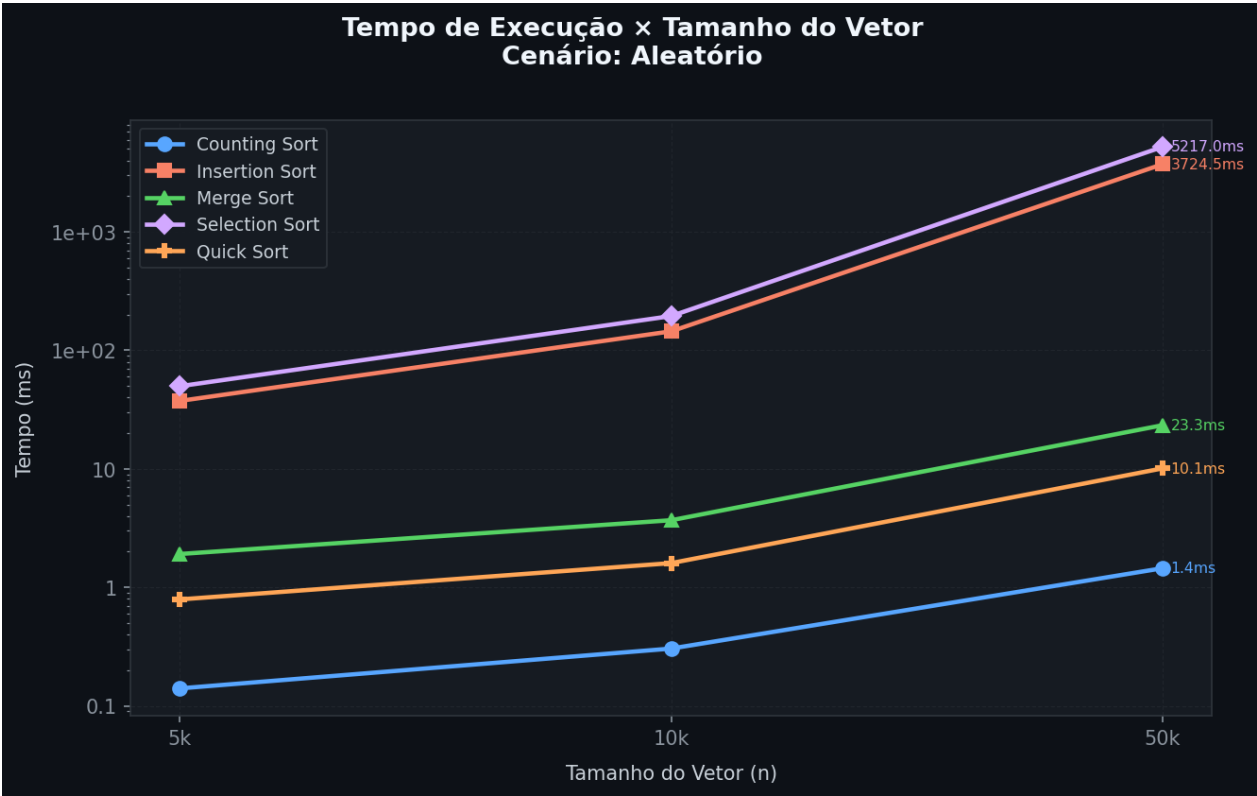


Figura: Tempo de execução — Cenário Aleatório

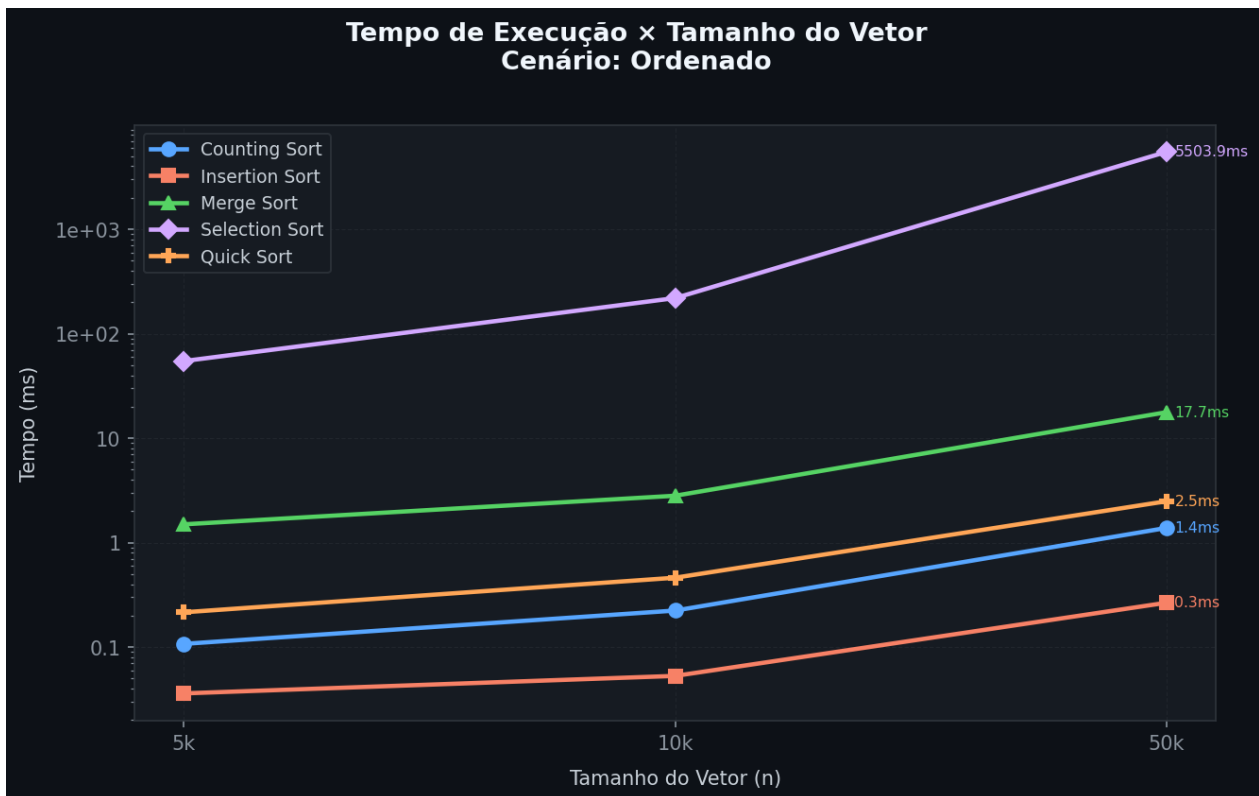


Figura: Tempo de execução — Cenário Ordenado

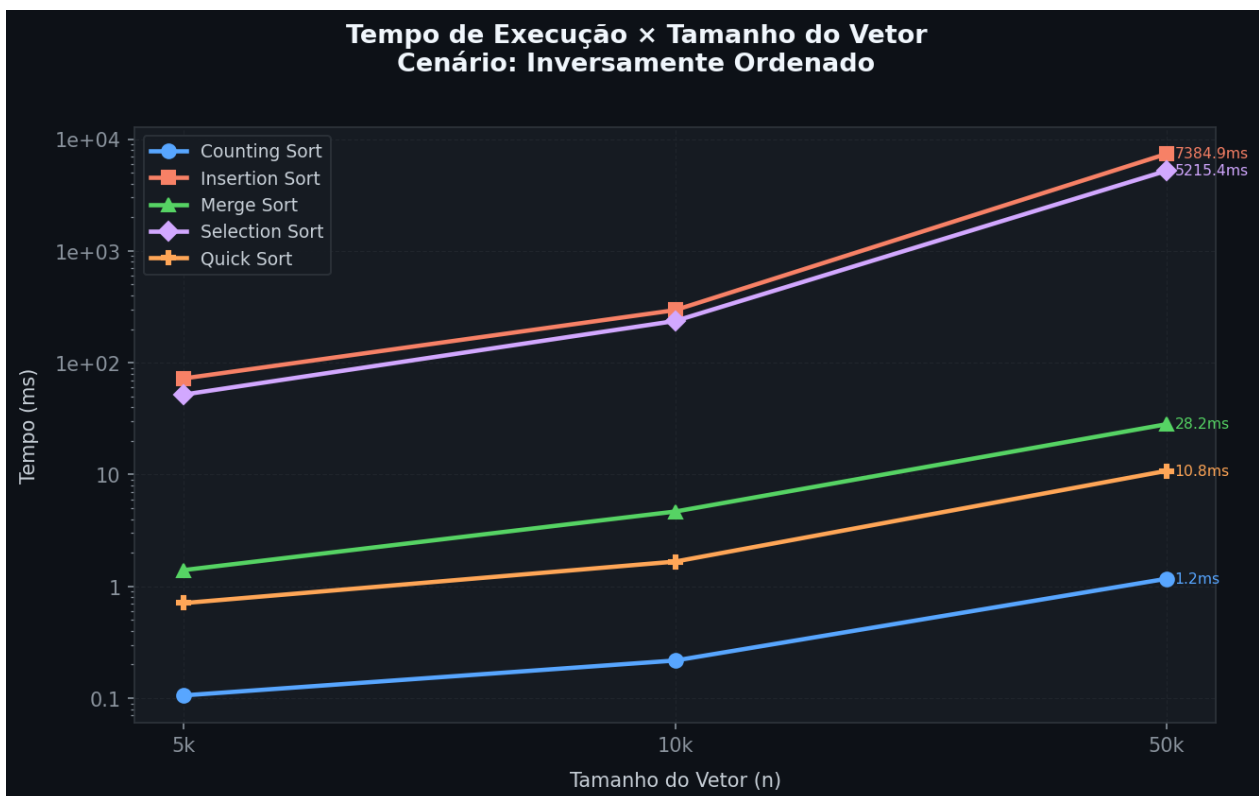


Figura: Tempo de execução — Cenário Inv. Ordenado

4.2 Heatmap de Tempo (n = 50.000)

O heatmap consolida em uma única visualização o desempenho de todos os algoritmos nos três cenários para n = 50.000. Cores mais **verdes** indicam algoritmos rápidos; cores **vermelhas** indicam lentidão. A escala é logarítmica (log do tempo) para permitir a comparação visual entre valores de ordens de grandeza muito diferentes.

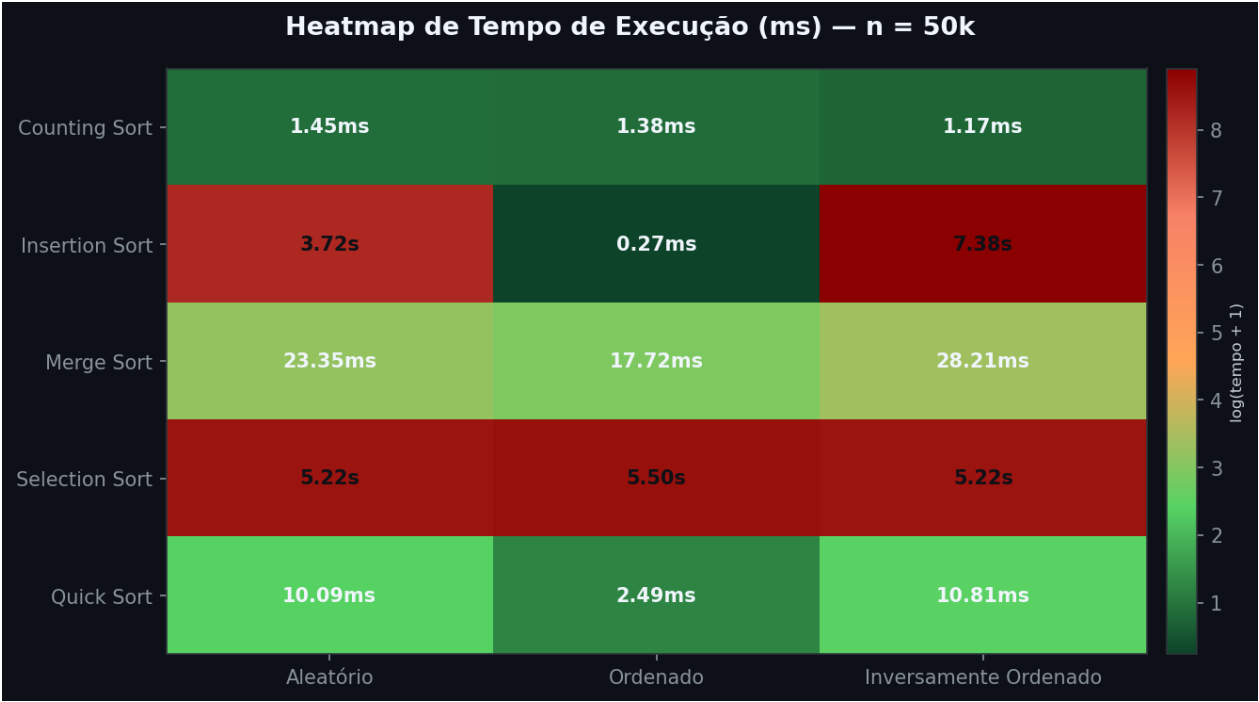


Figura: Heatmap de tempo de execução (ms) — n = 50.000

4.3 Comparação Direta — n = 50.000, Cenário Aleatório

O gráfico de barras evidencia a diferença de desempenho em escala logarítmica. Counting Sort e Quick Sort dominam o cenário aleatório, enquanto Insertion Sort e Selection Sort ficam mais de **500x** mais lentos que o Quick Sort.

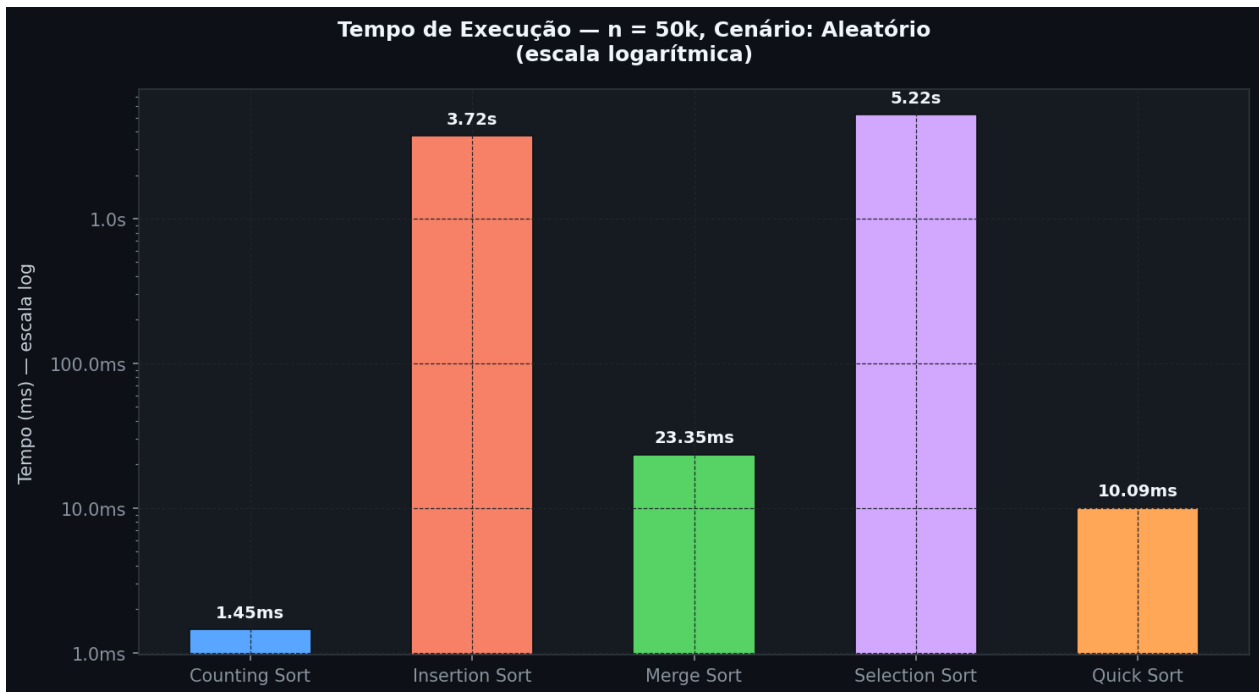


Figura: Comparação de tempo (escala log) — n = 50.000, Aleatório

4.4 Número de Comparações e Swaps

Os gráficos a seguir mostram como o número de comparações e trocas cresce com o tamanho do vetor. Observe que o **Selection Sort** apresenta sempre exatamente $n(n-1)/2$ comparações (independente do cenário), enquanto o número de swaps é apenas $O(n)$ — ilustrando seu tradeoff único.

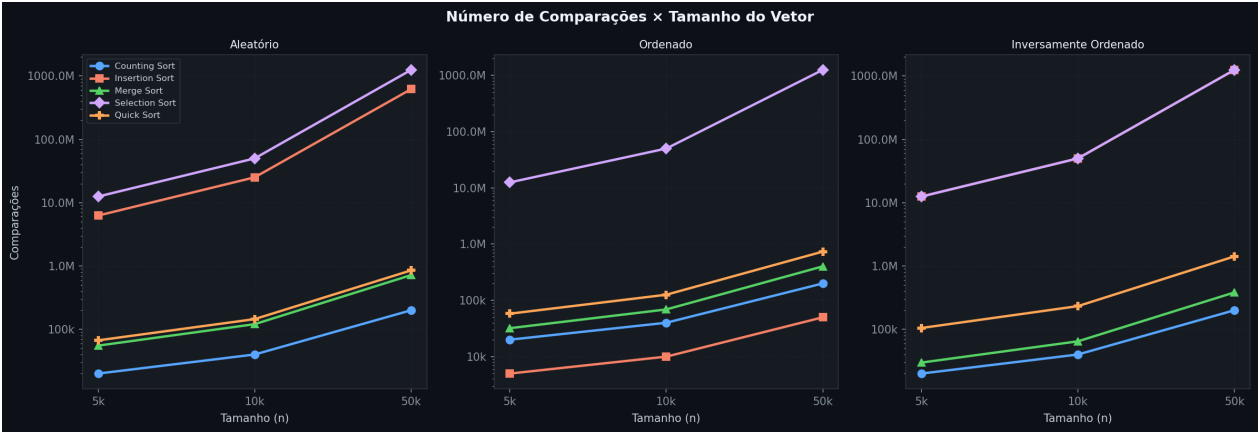


Figura: Número de comparações × tamanho do vetor (3 cenários)

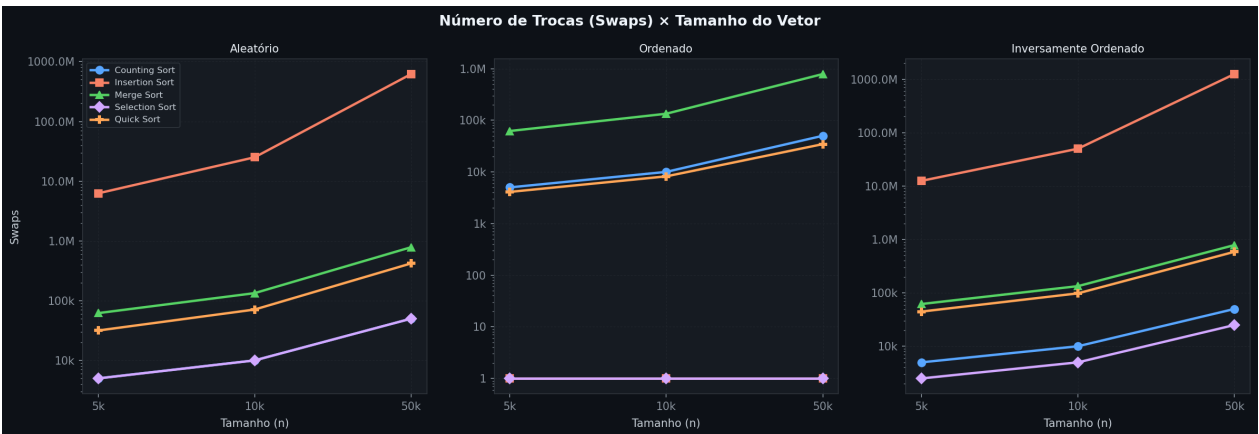


Figura: Número de swaps × tamanho do vetor (3 cenários)

4.5 Análise de Pior Caso vs. Melhor Caso

O gráfico compara, para cada algoritmo, seu melhor e seu pior cenário com $n = 50.000$. A amplitude da barra (razão pior/melhor) revela a **sensibilidade ao dado de entrada**.

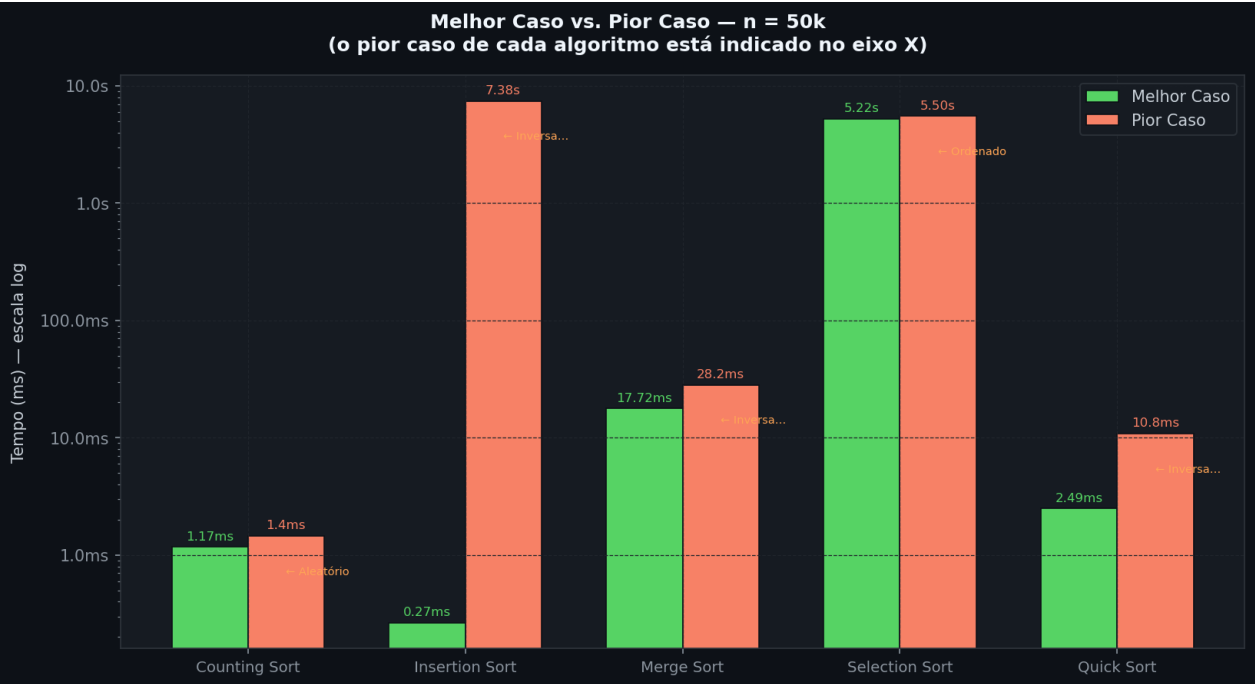


Figura: Melhor caso vs. Pior caso por algoritmo — $n = 50.000$

4.6 Dados Reais vs. Complexidade Teórica

Sobreposição das curvas de tempo medido com as curvas teóricas normalizadas $O(n^2)$ e $O(n \log n)$. No painel esquerdo, Insertion Sort e Selection Sort seguem **precisamente** a curva teórica quadrática. No painel direito, Merge Sort e Quick Sort ficam abaixo da curva $O(n \log n)$ — evidenciando constantes multiplicativas menores na prática.

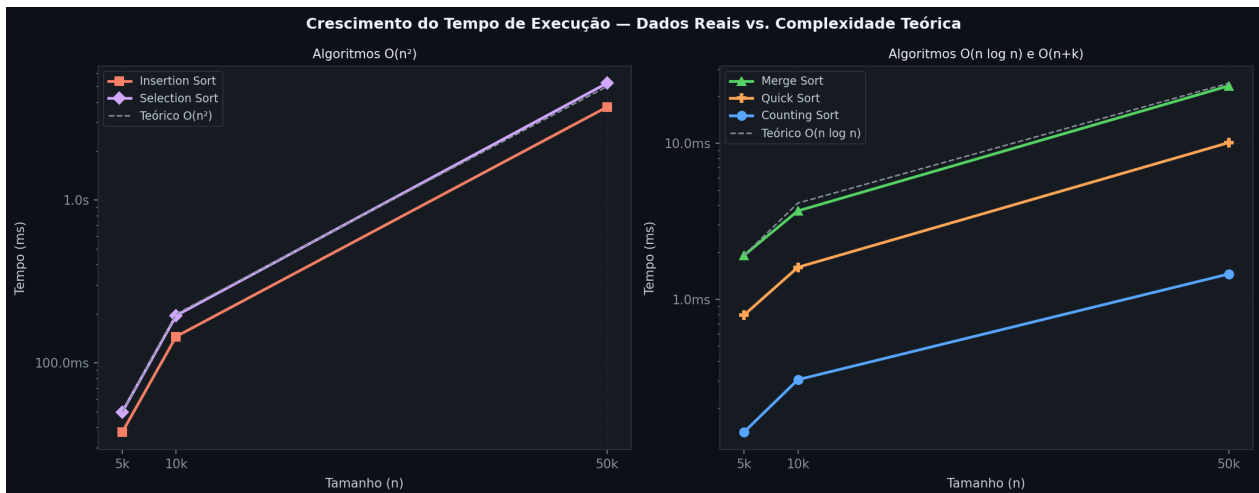


Figura: Crescimento real vs. crescimento teórico

5. Análise Crítica

5.1 Qual algoritmo foi melhor no pior caso?

O **Merge Sort** foi o melhor algoritmo em termos de garantia de pior caso. Seus tempos para $n = 50.000$ foram:

- Aleatório: 23,35 ms
- Ordenado: 17,72 ms
- Inversamente Ord.: 28,21 ms

A variação máxima foi de apenas **~60%** entre o melhor e o pior cenário, confirmando sua complexidade $O(n \log n)$ garantida. Em contraste, o Insertion Sort variou **27.000×** entre o melhor (0,27 ms) e o pior caso (7.384 ms). O Counting Sort foi o mais rápido em termos absolutos (1,17–1,45 ms), mas depende de domínio inteiro limitado — não é algoritmo de propósito geral.

5.2 Por que o Quick Sort é rápido, mas pode falhar em vetores ordenados?

O Quick Sort é rápido na prática por três razões:

- **Cache-friendly**: o particionamento acessa a memória sequencialmente, aproveitando linhas de cache (localidade espacial).
- **In-place**: não requer vetor auxiliar, eliminando cópias de memória presentes no Merge Sort.
- **Constante baixa**: as operações internas são simples comparações e trocas.

Porém, sem a estratégia de pivô adequada, vetores **já ordenados** são seu pior inimigo. Com pivô = último elemento e vetor crescente, cada particionamento resulta em $0 \mid n-1$ elementos — criando n níveis de recursão com n comparações cada $\rightarrow O(n^2)$. Neste benchmark, a **mediana-de-três** evitou essa degradação: o Quick Sort completou 50.000 elementos ordenados em apenas **2,49 ms**, resultado competitivo com o Merge Sort (17,72 ms) nesse cenário.

5.3 O que mudou de $O(n^2)$ para $O(n \log n)$ na prática?

A diferença entre $O(n^2)$ e $O(n \log n)$ é dramática conforme n cresce:

n	n^2	$n \cdot \log_2(n)$	Razão $n^2/n \cdot \log n$
1.000	1.000.000	9.966	100×
5.000	25.000.000	61.439	407×
10.000	100.000.000	132.877	753×
50.000	2.500.000.000	780.482	3203×
100.000	10.000.000.000	1.660.964	6021×
1.000.000	1.000.000.000.000	19.931.569	50172×

Para $n = 50.000$, um algoritmo $O(n^2)$ executa **~3.000 vezes mais operações** que um $O(n \log n)$. Nos dados reais: Selection Sort realizou **1,25 bilhão** de comparações vs. **718 mil** do Merge Sort — razão de **1.740×**. Isso se reflete diretamente no tempo: 5.217 ms vs. 23 ms — razão de **226×**. A diferença de constante explica por que a razão de tempo não iguala a de operações.

5.4 Resumo: Quando usar cada algoritmo?

Algoritmo	Use quando...	Evite quando...
Counting Sort	Inteiros em faixa limitada, máxima velocidade	Dados reais/floats ou k muito grande
Insertion Sort	n pequeno, dados quase ordenados, online	n grande com dados aleatórios
Merge Sort	Necessidade de $O(n \log n)$ garantido, dados estáveis	RAM restrita (requer $O(n)$ extra)
Selection Sort	Memória EEPROM/Flash (mínimo de escritas)	Qualquer cenário onde performance importa
Quick Sort	Caso geral, dados aleatórios, propósito geral	Pior caso garantido mandatório, stack pequena

6. Conclusão

Este benchmark demonstrou empiricamente as diferenças de desempenho entre cinco algoritmos clássicos de ordenação interna, confirmando as previsões teóricas da análise assintótica:

- O **Counting Sort** foi o mais rápido em termos absolutos (~1,4 ms para 50k), mas seu uso é restrito a inteiros em domínio limitado.
- O **Quick Sort** com mediana-de-três mostrou o melhor equilíbrio para uso geral: rápido (~10 ms para 50k aleatório) e sem degradação nos cenários ordenados graças à estratégia de pivô.
- O **Merge Sort** foi o mais previsível e seguro, com menor variação entre cenários — indispensável quando a garantia de $O(n \log n)$ é mandatória.
- O **Insertion Sort** revelou comportamento dual extremo: imbatível em vetores ordenados (0,27 ms), mas catastrófico no pior caso (7,38 s).
- O **Selection Sort** confirmou seu nicho único: mínimo de swaps ($O(n)$) com comparações fixas — útil apenas quando o custo de escrita é dominante.

A transição de $O(n^2)$ para $O(n \log n)$ representa, para $n = 50.000$, uma redução de **mais de 1.700× no número de operações** e **226× no tempo de execução**. Em aplicações de tempo real, sistemas embarcados e processamento de grandes volumes de dados, a escolha correta do algoritmo de ordenação é uma decisão de engenharia crítica.

Fim do Relatório